

TERRAFORM CHALLENGES 02

1. Create VPC
2. Create Internet gateway
3. Create Custom Route Table
4. Create Subnet
5. Associate subnet with Route Table
6. Create Security Group to allow port 22,80,443
7. Create a network interface with an ip in the subnet that was created in step 4
8. Assign an elastic IP to the network interface created in step 7
9. Create Ubuntu server and install/enable apache2

Step 1: Create VPC

What is VPC?

VPC (Virtual Private Cloud) is a logically isolated network in AWS where we create resources such as EC2 instances, subnets, route tables, etc.

Terraform Code

```
resource "aws_vpc" "main_vpc" {  
  cidr_block      = "10.0.0.0/16"  
  enable_dns_support = true  
  enable_dns_hostnames = true  
  tags = {  
    Name = "terraform-vpc"  
  }  
}
```

```
# =====  
# 2. CREATE VPC  
# =====  
  
resource "aws_vpc" "main_vpc" {  
  cidr_block      = "10.0.0.0/16"  
  enable_dns_support = true  
  enable_dns_hostnames = true  
  tags = {  
    Name = "terraform-vpc"  
  }  
}
```

Explanation

Configuration	Meaning
aws_vpc	Terraform AWS VPC resource
main_vpc	Terraform resource name
cidr_block	VPC IP range
10.0.0.0/16	Provides the VPC network range
enable_dns_support	Enables DNS resolution
enable_dns_hostnames	Allows DNS hostnames
tags	Gives a name to the VPC

Result

Terraform creates:

VPC

CIDR: 10.0.0.0/16

Name: terraform-vpc

Step 2: Create Internet Gateway

What is an Internet Gateway?

An **Internet Gateway (IGW)** allows communication between resources in a VPC and the internet.

Terraform Code

```
resource "aws_internet_gateway" "main_igw" {  
  vpc_id = aws_vpc.main_vpc.id  
  
  tags = {  
    Name = "terraform-igw"  
  }  
}
```

```
# =====
# 3. CREATE INTERNET GATEWAY
# =====

resource "aws_internet_gateway" "main_igw" {
  vpc_id = aws_vpc.main_vpc.id

  tags = {
    Name = "terraform-igw"
  }
}
```

Important

The Internet Gateway is attached to our VPC:

VPC

|

└─ Internet Gateway

Result

Terraform creates:

Internet Gateway

Name: terraform-igw

Attached VPC: terraform-vpc

Step 3: Create Custom Route Table

What is a Route Table?

A **route table** tells AWS where network traffic should go.

We create a route:

0.0.0.0/0 → Internet Gateway

This means:

Any traffic going outside the VPC should go through the Internet Gateway.

Terraform Code

```
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main_igw.id
  }
}
```

```

}

tags = {

  Name = "terraform-public-route-table"

}
}

```

```

# =====
# 4. CREATE CUSTOM ROUTE TABLE
# =====

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main_igw.id
  }

  tags = {
    Name = "terraform-public-route-table"
  }
}

```

Important Route

Destination	Target
-------------	--------

0.0.0.0/0	→ Internet Gateway
-----------	--------------------

Simple Explanation

EC2

|

| Internet traffic

↓

Route Table

|

| 0.0.0.0/0

↓

Internet Gateway

|

↓

Internet

Step 4: Create Subnet

What is a Subnet?

A **subnet** is a smaller network inside a VPC.

Our VPC is:

10.0.0.0/16

We create a subnet:

10.0.1.0/24

Terraform Code

```
resource "aws_subnet" "public_subnet" {  
    vpc_id          = aws_vpc.main_vpc.id  
    cidr_block      = "10.0.1.0/24"  
    availability_zone = "us-east-1a"  
    map_public_ip_on_launch = false  
  
    tags = {  
        Name = "terraform-public-subnet"  
    }  
}
```

```
# =====  
# 5. CREATE SUBNET  
# =====  
  
resource "aws_subnet" "public_subnet" {  
    vpc_id          = aws_vpc.main_vpc.id  
    cidr_block      = "10.0.1.0/24"  
    availability_zone = "us-east-1a"  
    map_public_ip_on_launch = false  
  
    tags = {  
        Name = "terraform-public-subnet"  
    }  
}
```

Explanation

Configuration	Meaning
vpc_id	Subnet belongs to our VPC
cidr_block	Subnet IP range
10.0.1.0/24	Subnet network
availability_zone	Subnet is created in us-east-1a

Configuration

Meaning

map_public_ip_on_launch = false Don't automatically assign public IP

Important

Even though automatic public IP assignment is disabled, we will later attach an **Elastic IP to the ENI**.

Step 5: Associate Subnet with Route Table

Creating a subnet and route table separately does **not** automatically connect them.

We need a **Route Table Association**.

Terraform Code

```
resource "aws_route_table_association" "public_association" {  
  subnet_id    = aws_subnet.public_subnet.id  
  route_table_id = aws_route_table.public_rt.id  
}
```

```
# =====  
# 6. ASSOCIATE SUBNET WITH ROUTE TABLE  
# =====  
  
resource "aws_route_table_association" "public_association" {  
  subnet_id    = aws_subnet.public_subnet.id  
  route_table_id = aws_route_table.public_rt.id  
}
```

Relationship

VPC

|

├— Internet Gateway

|

├— Route Table

| |

| └— 0.0.0.0/0 → IGW

|

└— Subnet

|

└— Route Table Association

Result

The subnet now uses our custom route table.

Step 6: Create Security Group

What is a Security Group?

A **Security Group** acts like a virtual firewall for EC2/network interfaces.

We need to allow:

22 → SSH

80 → HTTP

443 → HTTPS

Terraform Code

```
resource "aws_security_group" "web_sg" {  
  name      = "terraform-web-sg"  
  description = "Allow SSH HTTP and HTTPS"  
  vpc_id    = aws_vpc.main_vpc.id
```

```
  ingress {  
    description = "SSH"  
    from_port   = 22  
    to_port     = 22  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  ingress {  
    description = "HTTP"  
    from_port   = 80  
    to_port     = 80  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
ingress {  
  description = "HTTPS"  
  from_port   = 443  
  to_port     = 443  
  protocol    = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

```
egress {  
  description = "Allow all outbound traffic"  
  from_port   = 0  
  to_port     = 0  
  protocol    = "-1"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

```
tags = {  
  Name = "terraform-web-sg"  
}  
}
```

```

# =====
# 7. CREATE SECURITY GROUP
#   Allow SSH 22, HTTP 80, HTTPS 443
# =====

resource "aws_security_group" "web_sg" {
  name        = "terraform-web-sg"
  description = "Allow SSH HTTP and HTTPS"
  vpc_id      = aws_vpc.main_vpc.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow all outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-web-sg"
  }
}

```

Ports

Port Protocol Purpose

22	TCP	SSH
80	TCP	HTTP / Apache
443	TCP	HTTPS

Simple interview answer

A Security Group is a stateful virtual firewall associated with an EC2 instance or network interface. It controls inbound and outbound traffic.

Note: For production, SSH should generally be restricted to trusted IP ranges instead of 0.0.0.0/0.

Step 7: Create Network Interface

What is an ENI?

ENI = Elastic Network Interface

It is a virtual network interface that provides:

- Private IP address
- Security groups
- Network connectivity

Terraform Code

```

resource "aws_network_interface" "web_eni" {

  subnet_id    = aws_subnet.public_subnet.id

  private_ips  = ["10.0.1.10"]

  security_groups = [aws_security_group.web_sg.id]


  tags = {

    Name = "terraform-web-eni"

  }
}

```

```

# =====
# 8. CREATE NETWORK INTERFACE
# =====

resource "aws_network_interface" "web_eni" {
  subnet_id    = aws_subnet.public_subnet.id
  private_ips  = ["10.0.1.10"]
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "terraform-web-eni"
  }
}

```

Explanation

Our ENI gets:

Subnet:

10.0.1.0/24

Private IP:

10.0.1.10

Security Group:

terraform-web-sg

Architecture

VPC

|

└─ Subnet: 10.0.1.0/24

|

└─ ENI

|
├── Private IP: 10.0.1.10
└── Security Group

Step 8: Assign Elastic IP to Network Interface

What is an Elastic IP?

An **Elastic IP (EIP)** is a static public IPv4 address that can be associated with an AWS resource such as a network interface.

Terraform Code

```
resource "aws_eip" "web_eip" {  
    domain          = "vpc"  
    network_interface = aws_network_interface.web_eni.id  
    associate_with_private_ip = "10.0.1.10"  
  
    depends_on = [  
        aws_internet_gateway.main_igw  
    ]  
  
    tags = {  
        Name = "terraform-web-eip"  
    }  
}
```

```
# =====  
# 9. CREATE ELASTIC IP  
#   Attach it to the Network Interface  
# =====  
  
resource "aws_eip" "web_eip" {  
    domain          = "vpc"  
    network_interface = aws_network_interface.web_eni.id  
    associate_with_private_ip = "10.0.1.10"  
  
    depends_on = [  
        aws_internet_gateway.main_igw  
    ]  
  
    tags = {  
        Name = "terraform-web-eip"  
    }  
}
```

Relationship

Elastic IP

|

↓

ENI

|

↓

Private IP

10.0.1.10

The server will have:

Private IP → 10.0.1.10

Public IP → Elastic IP

Why EIP?

The Elastic IP provides a stable public IPv4 address.

Step 9: Create Ubuntu EC2 Server

Now we create the Ubuntu EC2 instance.

Terraform Code

```
resource "aws_instance" "web_server" {
  ami      = "ami-0c7217cdde317cfec"
  instance_type = "t3.micro"

  key_name = "devops.key"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index        = 0
  }

  user_data = <<-EOF
    #!/bin/bash

    apt-get update -y
```

```
apt-get install -y apache2
```

```
systemctl enable apache2
```

```
systemctl start apache2
```

```
echo "<h1>Hello from Terraform Ubuntu Apache Server</h1>" > /var/www/html/index.html  
EOF
```

```
tags = {  
  Name = "terraform-ubuntu-apache"  
}
```

```
depends_on = [  
  aws_eip.web_eip  
]  
}
```

```
# =====  
# 10. CREATE UBUNTU EC2 SERVER  
# AND INSTALL APACHE2  
# =====  
  
resource "aws_instance" "web_server" {  
  ami           = "ami-0c7217cdde317cfec"  
  instance_type = "t3.micro"  
  
  key_name = "devops.key"  
  
  network_interface {  
    network_interface_id = aws_network_interface.web_eni.id  
    device_index          = 0  
  }  
  
  user_data = <<-EOF  
  #!/bin/bash  
  
  apt-get update -y  
  apt-get install -y apache2  
  
  systemctl enable apache2  
  systemctl start apache2  
  
  echo "<h1>Hello from Terraform Ubuntu Apache Server</h1>" > /var/www/html/index.html  
  EOF  
  
  tags = {  
    Name = "terraform-ubuntu-apache"  
  }  
  
  depends_on = [  
    aws_eip.web_eip  
  ]  
}
```

EC2 Configuration

AMI

ami = "ami-0c7217cdde317cfec"

AMI means **A**mazon **M**achine **I**mage.

It provides the operating system image for the EC2 instance.

Here we are using an Ubuntu AMI.

Important: AMI IDs are region-specific and can change. Before running the code in your own account, verify that this AMI exists and is an Ubuntu AMI in us-east-1.

Instance Type

```
instance_type = "t3.micro"
```

This defines the compute capacity of the EC2 instance.

Key Pair

```
key_name = "devops.key"
```

This specifies the EC2 key pair used for SSH access.

Step 10: Attach Existing Network Interface to EC2

Instead of allowing EC2 to automatically create its primary network interface, our code attaches the ENI that we created earlier.

```
network_interface {  
    network_interface_id = aws_network_interface.web_eni.id  
    device_index          = 0  
}
```

What is device_index = 0?

0 means this ENI is the **primary network interface** of the EC2 instance.

Architecture:

EC2

|

└─ Primary ENI

|

└─ Private IP: 10.0.1.10

└─ Security Group

└─ Elastic IP

Step 11: Install Apache2 Using User Data

We use EC2 **User Data** to automatically install Apache when the instance launches.

```
#!/bin/bash
```

```
apt-get update -y
```

```
apt-get install -y apache2
```

```
systemctl enable apache2
```

```
systemctl start apache2
```

```
echo "<h1>Hello from Terraform Ubuntu Apache Server</h1>" > /var/www/html/index.html
```

Line-by-line explanation

1. Bash script

```
#!/bin/bash
```

Tells Linux to execute the script using Bash.

2. Update packages

```
apt-get update -y
```

Updates the Ubuntu package repository information.

3. Install Apache

```
apt-get install -y apache2
```

Installs the Apache web server.

4. Enable Apache

```
systemctl enable apache2
```

Makes Apache start automatically when the server boots.

5. Start Apache

```
systemctl start apache2
```

Starts Apache immediately.

6. Create web page

```
echo "<h1>Hello from Terraform Ubuntu Apache Server</h1>" > /var/www/html/index.html
```

Creates the website's index.html.

Step 12: Terraform Outputs

At the end, we define outputs so Terraform displays important information after deployment.

```
output "vpc_id" {
  value = aws_vpc.main_vpc.id
}

output "subnet_id" {
  value = aws_subnet.public_subnet.id
}

output "security_group_id" {
  value = aws_security_group.web_sg.id
}

output "network_interface_id" {
  value = aws_network_interface.web_eni.id
}

output "elastic_ip" {
  value = aws_eip.web_eip.public_ip
}

output "instance_id" {
  value = aws_instance.web_server.id
}

output "apache_url" {
  value = "http://${aws_eip.web_eip.public_ip}"
}
```

```

}

# =====
# 11. OUTPUTS
# =====

output "vpc_id" {
  value = aws_vpc.main_vpc.id
}

output "subnet_id" {
  value = aws_subnet.public_subnet.id
}

output "security_group_id" {
  value = aws_security_group.web_sg.id
}

output "network_interface_id" {
  value = aws_network_interface.web_eni.id
}

output "elastic_ip" {
  value = aws_eip.web_eip.public_ip
}

output "instance_id" {
  value = aws_instance.web_server.id
}

output "apache_url" {
  value = "http://${aws_eip.web_eip.public_ip}"
}

```

After terraform apply, you should get something similar to:

```

vpc_id          = "vpc-xxxxxxx"
subnet_id       = "subnet-xxxxxxx"
security_group_id = "sg-xxxxxxx"
network_interface_id = "eni-xxxxxxx"
elastic_ip      = "x.x.x.x"
instance_id     = "i-xxxxxxx"
apache_url      = http://x.x.x.x

```

Step 13: Terraform Commands

Create a file:

main.tf

Then run the following commands.

1. Initialize Terraform

terraform init

Purpose:

Downloads the required Terraform provider.

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform final chlng> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.64.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

2. Validate

terraform validate

Purpose:

Checks whether the Terraform configuration is syntactically valid.

Expected:

Success! The configuration is valid.

```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform final chlng> terraform validate
Success! The configuration is valid.
```

3. Format

terraform fmt

Purpose:

Formats Terraform code into standard Terraform style.

4. Plan

terraform plan

Purpose:

Shows what Terraform is going to create, modify, or destroy.

You should see resources such as:

aws_vpc

aws_internet_gateway

aws_route_table

aws_subnet

aws_route_table_association

aws_security_group

aws_network_interface

aws_eip

aws_instance

5. Apply

terraform apply

Terraform asks:

Do you want to perform these actions?

Enter:

yes

Terraform then creates the infrastructure.

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform final ch1ng> terraform apply
aws_vpc.main_vpc: Refreshing state... [id=vpc-0fb781a1ac5ee07b4]
aws_internet_gateway.main_igw: Refreshing state... [id=igw-02a90771385eaff47]
aws_subnet.public_subnet: Refreshing state... [id=subnet-0fd9ef951a56cf7b2]
aws_security_group.web_sg: Refreshing state... [id=sg-0040ecfae7a74c522]
aws_route_table.public_rt: Refreshing state... [id=rtb-01bad1e6300531645]
aws_network_interface.web_eni: Refreshing state... [id=eni-09a9a64865be91ac3]
aws_route_table_association.public_association: Refreshing state... [id=rtbassoc-03b9e036049a4141c]
aws_eip.web_eip: Refreshing state... [id=eipalloc-04ea20a477cd4fd9]
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
apache_url = "http://3.234.8.88"
elastic_ip = "3.234.8.88"
instance_id = "i-03290044c83a3a080"
network_interface_id = "eni-09a9a64865be91ac3"
security_group_id = "sg-0040ecfae7a74c522"
subnet_id = "subnet-0fd9ef951a56cf7b2"
vpc_id = "vpc-0fb781a1ac5ee07b4"
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform final ch1ng> []
```

Step 14: Verify Apache

After Terraform finishes, get the Elastic IP:

terraform output elastic_ip

Or:

terraform output apache_url

Example:

http://54.xx.xx.xx

Open the URL in your browser.

You should see:

Hello from Terraform Ubuntu Apache Server

Hello from Terraform Ubuntu Apache Server

I write this script for task:

```
# =====
```

```
# 1. PROVIDER
```

```
# =====
```

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
    }  
  }  
}
```

```
provider "aws" {  
  region = "us-east-1"  
}
```

```
# =====
```

```
# 2. CREATE VPC
```

```
# =====
```

```
resource "aws_vpc" "main_vpc" {  
  cidr_block      = "10.0.0.0/16"  
  enable_dns_support = true  
  enable_dns_hostnames = true
```

```
tags = {  
  Name = "terraform-vpc"  
}  
}
```

```
# =====  
# 3. CREATE INTERNET GATEWAY  
# =====
```

```
resource "aws_internet_gateway" "main_igw" {  
  vpc_id = aws_vpc.main_vpc.id
```

```
tags = {  
  Name = "terraform-igw"  
}  
}
```

```
# =====  
# 4. CREATE CUSTOM ROUTE TABLE  
# =====
```

```
resource "aws_route_table" "public_rt" {  
  vpc_id = aws_vpc.main_vpc.id  
  
  route {  
    cidr_block = "0.0.0.0/0"  
    gateway_id = aws_internet_gateway.main_igw.id  
  }  
}
```

```
tags = {  
  Name = "terraform-public-route-table"
```

```

    }
}

# =====

# 5. CREATE SUBNET

# =====

resource "aws_subnet" "public_subnet" {
    vpc_id      = aws_vpc.main_vpc.id
    cidr_block   = "10.0.1.0/24"
    availability_zone = "us-east-1a"
    map_public_ip_on_launch = false

    tags = {
        Name = "terraform-public-subnet"
    }
}

# =====

# 6. ASSOCIATE SUBNET WITH ROUTE TABLE

# =====

resource "aws_route_table_association" "public_association" {
    subnet_id      = aws_subnet.public_subnet.id
    route_table_id = aws_route_table.public_rt.id
}

# =====

# 7. CREATE SECURITY GROUP

#   Allow SSH 22, HTTP 80, HTTPS 443

# =====

```

```
resource "aws_security_group" "web_sg" {  
  name      = "terraform-web-sg"  
  description = "Allow SSH HTTP and HTTPS"  
  vpc_id    = aws_vpc.main_vpc.id
```

```
  ingress {  
    description = "SSH"  
    from_port   = 22  
    to_port     = 22  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }
```

```
  ingress {  
    description = "HTTP"  
    from_port   = 80  
    to_port     = 80  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }
```

```
  ingress {  
    description = "HTTPS"  
    from_port   = 443  
    to_port     = 443  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }
```

```
  egress {
```

```
description = "Allow all outbound traffic"

from_port = 0

to_port = 0

protocol = "-1"

cidr_blocks = ["0.0.0.0/0"]

}
```

```
tags = {
  Name = "terraform-web-sg"
}
}
```

```
# =====
# 8. CREATE NETWORK INTERFACE
# =====
```

```
resource "aws_network_interface" "web_eni" {
  subnet_id = aws_subnet.public_subnet.id
  private_ips = ["10.0.1.10"]
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "terraform-web-eni"
  }
}
```

```
# =====
# 9. CREATE ELASTIC IP
# Attach it to the Network Interface
# =====
```

```

resource "aws_eip" "web_eip" {
  domain          = "vpc"
  network_interface = aws_network_interface.web_eni.id
  associate_with_private_ip = "10.0.1.10"

  depends_on = [
    aws_internet_gateway.main_igw
  ]

  tags = {
    Name = "terraform-web-eip"
  }
}

# =====
# 10. CREATE UBUNTU EC2 SERVER
#   AND INSTALL APACHE2
# =====

resource "aws_instance" "web_server" {
  ami          = "ami-0c7217cdde317cfec"
  instance_type = "t3.micro"

  key_name = "devops.key"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index         = 0
  }

  user_data = <<-EOF

```

```
#!/bin/bash
```

```
apt-get update -y
```

```
apt-get install -y apache2
```

```
systemctl enable apache2
```

```
systemctl start apache2
```

```
echo "<h1>Hello from Terraform Ubuntu Apache Server</h1>" > /var/www/html/index.html
```

```
EOF
```

```
tags = {  
  Name = "terraform-ubuntu-apache"  
}
```

```
depends_on = [  
  aws_eip.web_eip  
]  
}
```

```
# =====
```

```
# 11. OUTPUTS
```

```
# =====
```

```
output "vpc_id" {  
  value = aws_vpc.main_vpc.id  
}
```

```
output "subnet_id" {  
  value = aws_subnet.public_subnet.id  
}
```

```
output "security_group_id" {  
    value = aws_security_group.web_sg.id  
}
```

```
output "network_interface_id" {  
    value = aws_network_interface.web_eni.id  
}
```

```
output "elastic_ip" {  
    value = aws_eip.web_eip.public_ip  
}
```

```
output "instance_id" {  
    value = aws_instance.web_server.id  
}
```

```
output "apache_url" {  
    value = "http://${aws_eip.web_eip.public_ip}"  
}
```